

Documento complementar com orientações para o desenvolvimento do projeto
da disciplina Desenvolvimento de Sistemas Backend

Sistema de Controle de Planos de Operadora Especificação do Sistema

Domínio do problema

Considere uma startup que trabalha criando serviços para pequenas e grandes operadoras de internet.



Ícones de aplicações.

Neste modelo de negócio, os provedores, periodicamente, precisam verificar se a assinatura de um plano continua válida para poder continuar fornecendo o serviço. Em função disso, o sistema tem de ser capaz de responder rapidamente se uma determinada assinatura continua válida. Quando um usuário assina um plano, deve ser gerado um código (código do plano). Esse código, juntamente com o código do cliente, é utilizado pela operadora e seus serviços para confirmar se um plano continua ativo ou não para determinado cliente.

Para fazer toda esta gestão, startups que trabalham nesse modelo de negócio necessitam de um software de apoio que tenha as seguintes funcionalidades:

- Cadastrar/editar/listar a base de planos que disponibiliza
- Cadastrar/editar/listar a base de clientes
- Cadastrar a assinatura de um ou mais planos por parte de um cliente
- Atualizar o valor do custo mensal de um plano
- Responder se um par cliente/plano (representando uma assinatura) continua válido
- Listar as assinaturas de um cliente
- Listar os assinantes de um plano
- Receber a notificação de pagamento de uma assinatura de um plano (enviado pelo banco conveniado) e atualizar a data de fidelidade da assinatura

O objetivo deste trabalho é desenvolver o módulo “backend” de um sistema de apoio para empresas que disponibilizem seus planos e serviços neste modelo de negócio.

Considerando o prazo disponível para o desenvolvimento desta tarefa, faremos uma simplificação importante: cada instância deste sistema irá atender uma única operadora. Isso significa que todos os planos e clientes cadastrados no sistema pertencem a uma única empresa.

Requisitos não funcionais

Como o objetivo deste trabalho é desenvolver um “backend” dentro de certos parâmetros iremos definir, na sequência, o conjunto de entidades de domínio a serem utilizadas, bem como a arquitetura do sistema em termos de um serviço principal e um conjunto de microserviços.

Entidades de domínio

Considerando a descrição dos objetivos e funcionalidades do sistema foram definidas as seguintes entidades de domínio:

- Plano: modela um plano comercializado pela empresa no sistema de assinatura
- Cliente: pessoa interessada em assinar os planos disponibilizados pela empresa
- Assinatura: modela a relação entre um plano e um cliente
- Pagamento: modela o pagamento de uma determinada assinatura em um determinado mês

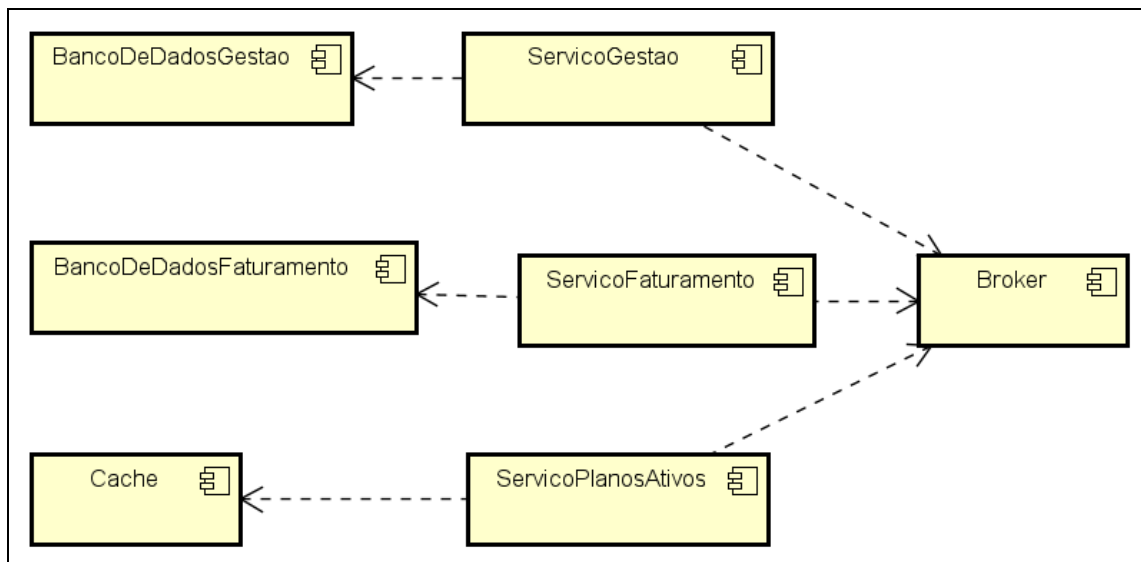
A tabela que segue detalha os atributos de cada uma das entidades de domínio.

Entidade	Atributos		
Plano	Atributo	Descrição	Tipo
	codigo	Código identificador do plano	Inteiro longo
	nome	Nome do plano, visível a todos os clientes	Texto
	custoMensal	Valor da assinatura mensal	Número de ponto flutuante
	data	Data de última modificação do plano	Data em formato ano, mês e dia
	descricao	Descrição do plano, com as características do mesmo, por exemplo: se tem internet móvel ou fibra, plano de TV incluso ou não, telefone fixo incluso ou não, etc.	Texto
Cliente	Atributo	Descrição	Tipo
	codigo	Código identificador do cliente	Inteiro longo
	nome	Nome do cliente	Texto
	email	Email do cliente	Texto
Assinatura	Atributo	Descrição	Tipo
	codigo	Código da assinatura	Inteiro longo
	codPlano	Código do plano assinado	Inteiro longo
	codCli	Código do cliente	Inteiro longo
	inicioFidelidade	Início da fidelidade do plano	Data em formato ano, mês e dia
	fimFidelidade	Fim da fidelidade do plano	Data em formato ano, mês e dia
	dataUltimoPagamento	Data em que o último pagamento foi realizado (para o serviço de internet continuar ativo, é requerido pagamento a cada 30 dias, sem tolerância de atraso)	Data em formato ano, mês e dia

	custoFinal	Custo final para o cliente dado o desconto aplicado devido à fidelidade, quando a fidelidade acaba o cliente passa a pagar o custo mensal do plano	Número de ponto flutuante
	descricao	Descreve quais critérios foram utilizados para aplicar o valor de fidelidade que está em "custoFinal"	Texto
Pagamento	Atributo	Descrição	Tipo
	codigo	Identificador único do pagamento	Inteiro longo
	codAss	Código da assinatura que está sendo paga	Inteiro longo
	valorPago	Valor pago	Número de ponto flutuante
	dataPagamento	Data em que o pagamento foi efetivado	Data em formato ano, mês e dia

Arquitetura de serviços

Visando explorar todos os aspectos desenvolvidos ao longo do curso, será proposta uma arquitetura mista compreendendo serviços e microserviços. Um serviço principal será responsável pelas seguintes funcionalidades: cadastramento e manutenção dos dados relativos a clientes, planos e assinaturas. Um microserviço será responsável por manter o registro dos pagamentos efetuados (este serviço será integrado com o serviço de pagamento da operadora para receber as requisições). Por questões de performance, um terceiro microserviço será capaz de responder rapidamente para os serviços se uma determinada assinatura continua ativa. A figura que segue apresenta um diagrama de componentes que representa essa arquitetura.



Arquitetura do sistema em nível de componentes.

O diagrama acima foi criado na ferramenta Astah, mas para o desenvolvimento do projeto, utilize a ferramenta de sua preferência. Pode-se, por exemplo, utilizar ferramentas que geram o diagrama a partir de texto, este é o código textual do diagrama apresentado acima:

@startuml

component BancoDeDadosFaturamento
component BancoDeDadosGestao
component Broker
component Cache
component ServicoFaturamento
component ServicoGestao
component ServicoPlanosAtivos

ServicoGestao .up.> BancoDeDadosGestao
ServicoGestao ..> Broker
ServicoFaturamento .up.> BancoDeDadosFaturamento
ServicoFaturamento ..> Broker
ServicoPlanosAtivos .up.> Cache
ServicoPlanosAtivos ..> Broker

@enduml

Este texto do diagrama pode ser utilizado no editor online <https://editor.plantuml.com/> e a documentação completa está disponível em <https://plantuml.com/>.

O serviço chamado de “ServicoGestao” será o módulo principal do nosso *backend*. Ele deverá ser o responsável por todas as operações de manutenção dos cadastros (clientes, planos e assinaturas) bem como pelas operações relativas à cobrança, tais como, atualizar o preço das assinaturas, atualizar a data de fidelidade etc. Sempre que uma assinatura for cadastrada, entra em período de fidelidade, pagando o valor com desconto. A fidelidade é de um ano (365 dias) a partir do dia da contratação. A extensão do período de fidelidade da assinatura se dá mediante a vontade do cliente a partir de uma comunicação com atendente que deve lhe ofertar um novo valor promocional e cadastrar no sistema. Podendo o período de fidelidade ser alterado a qualquer momento.

O microsserviço “ServicoFaturamento” mantém uma base dados com todos os pagamentos efetuados. Sempre que receber um pagamento, deve ser armazenado no banco e um evento assíncrono deve ser gerado para notificar os interessados de que um pagamento foi efetuado.

O “ServicoGestao” já é capaz de responder se uma determinada assinatura é válida ou não. Entretanto, como a demanda por esta informação é muito grande (todos os serviços periodicamente necessitam fazer essa consulta), foi projetado um microsserviço adicional visando garantir a performance do sistema. Desta forma, o microsserviço “ServicoPlanosAtivos” será responsável por responder, de forma rápida, se uma determinada assinatura de plano é ativa ou não. Ele será demandado, tipicamente pelos serviços de terceiros, sempre que estes tiverem necessidade de confirmar se devem continuar respondendo ou se devem bloquear o acesso por falta de pagamento da assinatura. Cada vez que for demandado, este microsserviço deve consultar sua “cache” (esta cache pode ser em memória ou não, dependendo da decisão do desenvolvedor) interna verificando se já possui a informação relativa àquela assinatura. Caso não disponha, deverá perguntar para o “ServicoGestao” e, então, registrar em sua “cache” a informação para consultas futuras.

Tanto o “ServicoGestao” quanto o “ServicoPlanosAtivos” consomem o evento que notifica que um pagamento foi efetuado. Com essa informação o “ServicoGestao”, usando suas regras de negócio, deverá atualizar a validade da assinatura. Já o “ServicoPlanosAtivos” deverá remover da sua “cache” a

entrada correspondente a assinatura paga. Desta forma, da próxima vez que for solicitado a respeito dessa assinatura, irá solicitar para o “ServicoGestao” a informação atualizada evitando inconsistências.

Requisitos funcionais

Nesta seção, iremos detalhar todos os endpoints de cada um dos serviços listados anteriormente. Para cada endpoint será detalhada a rota, os parâmetros recebidos e o JSON resposta.

ServicoGestao

Por uma questão de dimensionamento de escopo, iremos omitir desse trabalho todos os endpoints relativos ao cadastramento/manutenção de clientes e cadastramento/manutenção de planos. Para que o sistema possa ser testado a contento, deve ser previsto um script (“seeding”) de inicialização capaz de popular o banco de dados com pelo menos 10 clientes e 5 planos diferentes. Embora o sistema deva incluir os endpoints relativos ao gerenciamento de assinaturas, como será visto na sequência, o mesmo script deverá prever o cadastramento prévio de pelo menos 5 assinaturas de planos.

A seguir, são detalhados os endpoints do serviço “ServicoGestao”.

Endpoint (FASE 1): GET /gestao/clientes

Descrição	Listar todos os clientes cadastrados
Parâmetros de entrada	Nenhum
JSON resposta	Array de objetos com todos os atributos da entidade

Endpoint (FASE 1): GET /gestao/planos

Descrição	Listar todos os planos cadastrados
Parâmetros de entrada	Nenhum
JSON resposta	Array de objetos com todos os atributos da entidade

Endpoint (FASE 1): POST /gestao/assinaturas

Descrição	Criar uma assinatura
Corpo da requisição	{código do cliente, código do plano, custo final e descrição}
JSON resposta	Objeto com todos os atributos da entidade da assinatura

Endpoint (FASE 1): PATCH /gestao/planos/:idPlano

Descrição	Atualizar o custo mensal do plano
Corpo da requisição	{custoMensal}
JSON resposta	Objeto com todos os atributos da entidade do plano

Endpoint (FASE 1): GET /gestao/assinaturas/{tipo}

Descrição	Retorna a lista com todas as assinaturas de um determinado tipo
Parâmetros de entrada	Tipo = [TODOS ATIVOS CANCELADOS]
JSON resposta	Array de objetos [{código assinatura, código cliente, código plano, data de início, data de fim, status}, ...] Se a assinatura está ativa, a data de encerramento é posterior a data atual Valores para status = [ATIVO CANCELADO]

Endpoint (FASE 1): GET /gestao/assinaturascliente/:codcli

Descrição	Retorna a lista das assinaturas do cliente informado
Parâmetros de entrada	codcli: código do cliente
JSON resposta	Array de objetos [{código assinatura, código cliente, código plano, data de início, data de fim, status}, ...] Valores para status = [ATIVO CANCELADO]

Endpoint (FASE 1): GET / gestao/assinaturasplano/:codplano

Descrição	Retorna a lista de assinaturas de um plano
Parâmetros de entrada	codplano: código do plano
JSON resposta	Array de objetos [{código assinatura, código cliente, código plano, data de início, data de fim, status}, ...] Valores para status = [ATIVO CANCELADO]

Evento (observar) (FASE 2): PagamentoPlanoServicoGestao

Descrição	Observador do evento de pagamento de um plano
Corpo do evento	dia, mês e ano: compõem a data do pagamento código assinatura: identificador da assinatura a que corresponde o pagamento valorPago: valor que foi pago pela assinatura na data indicada

ServicoFaturamento

Este serviço possui uma base de dados própria usada única e exclusivamente para armazenar o registro de todos os pagamentos efetuados.

A seguir, são detalhados os endpoints do serviço “ServicoFaturamento”.

Endpoint (FASE 2): POST /registrarpagamento

Descrição	Solicita o registro de um pagamento
Parâmetros de entrada	dia, mês e ano: compõem a data do pagamento código assinatura: identificador da assinatura a que corresponde o pagamento

	valorPago: valor que foi pago pela assinatura na data indicada
JSON resposta	Nenhum

Evento (gerar) (FASE 2): PagamentoPlanoServicoGestao

Descrição	Evento de notificação de um pagamento efetuado para o ServicoGestao
Corpo do evento	dia, mês e ano: compõem a data do pagamento código assinatura: identificador da assinatura a que corresponde o pagamento valorPago: valor que foi pago pela assinatura na data indicada

Evento (gerar) (FASE 2): PagamentoPlanoServicoPlanosAtivos

Descrição	Evento de notificação de um pagamento efetuado para o ServicoPlanosAtivos
Corpo do evento	dia, mês e ano: compõem a data do pagamento código assinatura: identificador da assinatura a que corresponde o pagamento valorPago: valor que foi pago pela assinatura na data indicada

ServicoPlanosAtivos

Este microserviço deve ser capaz de manter uma “cache” de assinaturas de planos ativos. Deve ser capaz de responder de forma síncrona se uma determinada assinatura permanece ativa ou não. Como este serviço será demandado pelos serviços com instancias ativas das assinaturas de planos, espera-se que este módulo tenha alta demanda. Por essa razão deve ser programado como um microserviço do qual podemos ter várias instâncias.

A seguir, são detalhados os endpoints do serviço “ServicoPlanosAtivos”.

Endpoint (FASE 2): GET /planosativos/:codass

Descrição	Retorna se plano questionado permanece ativo
Parâmetros de entrada	codass: código da assinatura
JSON resposta	Booleano: true ou false

Evento (observar) (FASE 2): PagamentoPlanoServicoPlanosAtivos

Descrição	Observador do evento de pagamento
Corpo do evento	dia, mês e ano: compõem a data do pagamento código assinatura: identificador da assinatura a que corresponde o pagamento valorPago: valor que foi pago pela assinatura na data indicada

Entregáveis

- Arquivos do projeto para ser executado (sem a pasta node_modules)
- Documentação (PDF e arquivo do Postman)
- Caso a assinatura dos endpoints da sua aplicação seja diferente do template do Postman disponibilizado (template.postman_collection.json), certifique-se de atualizar o Postman de acordo com os endpoints da sua aplicação